SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I KANDATH HANEESH *(*192324084*) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Kandath Haneesh**

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Web Concepts	25%	Accurately explains HTTP, DOM, HTML and browser-server concepts in every answer	Explains concepts correctly with minor gaps	Partial understanding; misses key concepts	Misunderstands or fails to explain the concepts
Analysis of Errors and Issues	25%	Identifies all errors and issues with strong reasoning and justification	Identifies most errors with reasonable justification	Identifies few errors; weak reasoning	Fails to identify errors; no meaningful analysis
Application of Code and Standards	20%	Effectively applies HTML/HTTP standards; corrected code is valid and complete	Applies standards with minor errors in corrected code	Limited application; corrections are incomplete	Unable to apply standards or correct the code
Conclusion and Best Practices	20%	Draws clear conclusions with best-practice recommendations and implications	Provides valid conclusions with moderate recommendations	Conclusions are vague or weak	No clear or valid conclusions
Clarity and Presentation of Output	10%	Presents answers clearly with proper structure, code blocks and output screenshots	Generally clear with minor issues in structure or output	Lacks clarity or proper structure; few outputs	Poor presentation; no outputs shown

ANSWERS

Name: Kandath Haneesh **Reg No:** 192324084

Note on outputs: the screenshots in this document are real outputs produced by running the given header, DOM and HTML snippets on a local test setup (curl against a local test server, an HTML5 parser for the DOM and a browser engine for rendering).

Question 1: HTTP Response Analysis

Given response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Incorrect or invalid fields

- **Content-Length: -150** – invalid. The value must be a non-negative integer giving the body size in bytes.
- **Date: Mon, 01 Apr 2026** – inconsistent. 1 April 2026 falls on a **Wednesday**, so the day name does not match the date.
- **Content-Type: text/html** – valid, but it does not declare a character set (recommended: charset=UTF-8).
- The status line, Connection header and the header layout are valid. Each header must be on its own line and the block must end with a blank line.

2. Issue with Content-Length and its impact

Content-Length tells the client exactly how many bytes of body to read. This is how it knows where the message ends, especially on a persistent (keep-alive) connection. A negative number is meaningless, so the client cannot work out the message boundary. Impact: the client rejects the response (as curl does in Figure 1), the page does not load, or the connection is closed. Lenient clients that guess the length may truncate the page or hang while waiting for data. On keep-alive connections a wrong length can also corrupt the next response and can be abused for request-smuggling attacks.

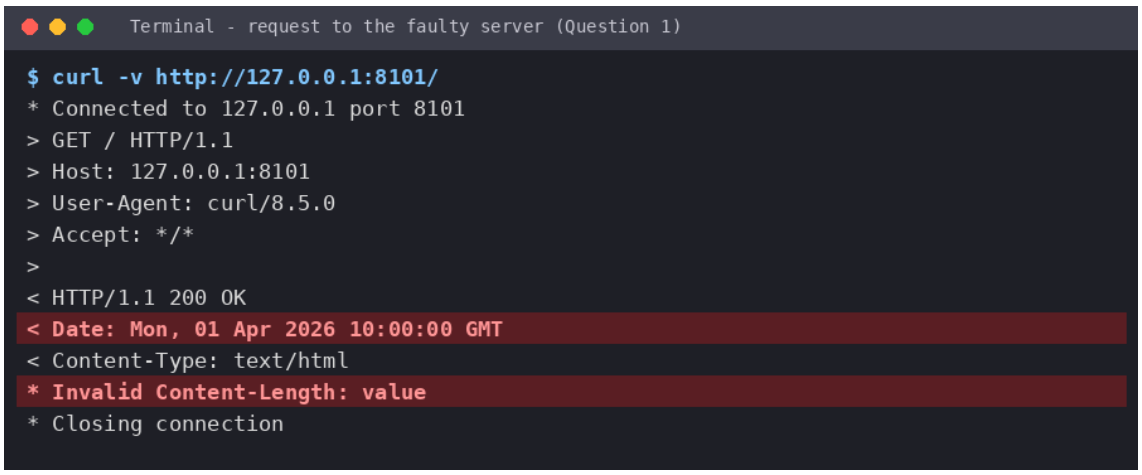


Figure 1: curl rejects the response – "Invalid Content-Length: value"

3. Does the status code match the response correctness?

No. The status line says **200 OK** (success), but the message itself is malformed and unusable, so the client treats it as a failure. The code does not reflect the real outcome. If the server cannot build a valid response, it should return **500 Internal Server Error**; otherwise, it must send a correct header with 200 OK.

4. Corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 31
Connection: keep-alive
```

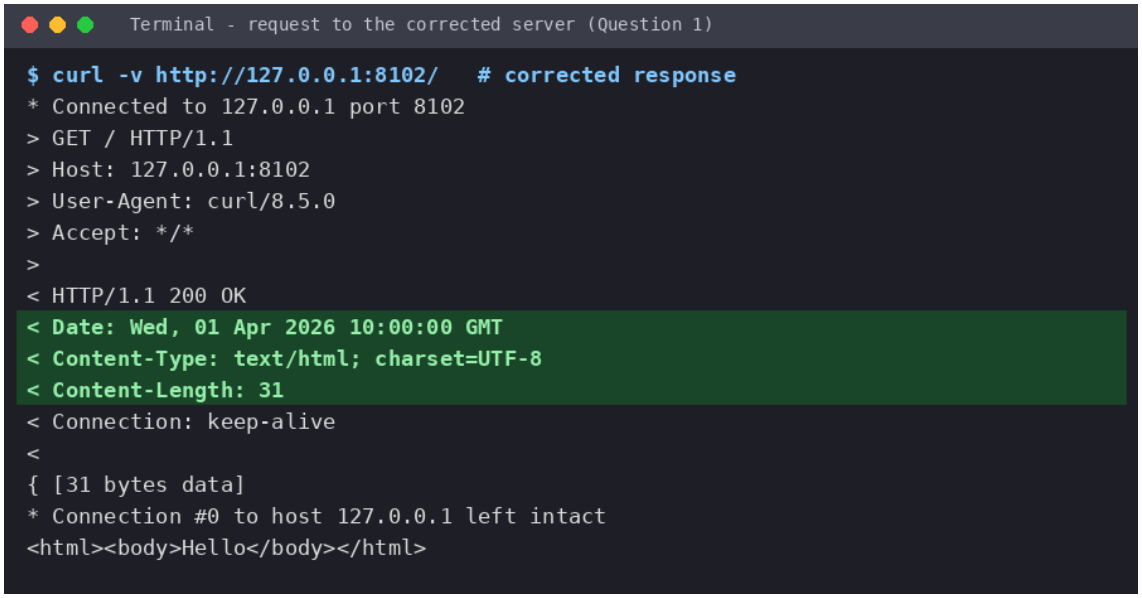


Figure 2: the corrected response is accepted and the 31-byte body is received

Conclusion: the header is invalid mainly because of the negative Content-Length (plus the wrong weekday). A 200 status cannot make up for a malformed message, so every field must be validated before sending.

Question 2: DOM Structure Analysis

```
<html>
 <head>
  <title>TestPage</title>
 </head>
 <body>
  <div>
   <p>HelloWorld
  </div>
 </body>
</html>
```

1. Missing or improperly closed elements

- The ** ** element is never closed – the closing tag ** ** is missing (the only real structural error).
- Not an error but recommended: a **<!DOCTYPE html>** declaration, a **lang** attribute on <html> and a **<meta charset>** tag in the head.

2. How the browser interprets the structure

Browsers do not stop on errors. The HTML5 parsing algorithm repairs the markup. When the parser reaches the end tag ** ** while a is still open, it closes the paragraph first and then the div. The DOM therefore ends up well-formed: html → head → title, and html → body → div → p → text “HelloWorld”. Because there is no DOCTYPE, the page is displayed in quirks mode.

DOM constructed by the parser (Q2)

```
DOM tree built by an HTML5 parser (html5lib):
<html>
  <head>
    <title>
      TestPage
    </title>
  </head>
  <body>
    <div>
      <p>
        HelloWorld
      </p>
    </div>
  </body>
</html>
```

Figure 3: DOM built by the parser – the missing was added automatically

3. Impact on rendering and layout

In this small page there is no visible difference: the text “HelloWorld” appears as a block paragraph with the default margins inside the div (Figure 4). The risk is in larger pages: an unclosed is closed implicitly by the next block element, so the DOM can differ from what the author intended. This can break CSS selectors, JavaScript DOM access and the layout, and different browsers or quirks mode may handle it slightly differently.



Figure 4: browser output for the original markup – renders normally, error hidden

4. Corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld</p>
    </div>
  </body>
</html>
```

Conclusion: the browser silently fixes the missing </p>, but relying on error recovery is risky. Always close elements and validate the page (for example with the W3C validator).

Question 3: GET vs POST in Login System

1. Differences in data transmission

Aspect	GET	POST
Where data goes	Appended to the URL as a query string (?user=admin&pass=1234)	Sent inside the HTTP request body
Visibility	Visible in address bar, history, bookmarks, logs	Not shown in the URL
Data size	Limited by URL length (about 2–8 KB in practice)	No practical limit (files, large forms)
Caching / bookmarking	Cached and bookmarkable	Not cached or bookmarked by default
Purpose	Reading or fetching data (safe, idempotent)	Sending data / changing state (creating a session)
Security level	Low	Moderate (needs HTTPS to be truly secure)

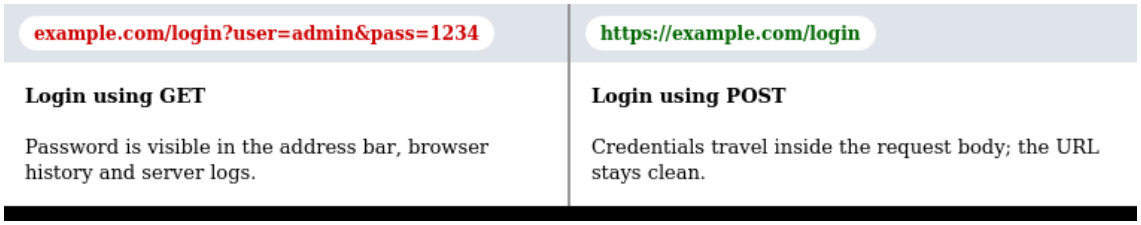


Figure 5: the same login with GET (password in URL) and POST (clean URL)



Figure 6: the server log shows the password in GET's request line; with POST it is only in the body

2. Which method is more suitable for login, and why

POST is the suitable method. A login submits sensitive credentials and creates a session on the server, which is a state-changing action. POST keeps the credentials out of the URL, so they are not stored in history, bookmarks or access logs. POST by itself does not encrypt data: the body is readable on the network unless the site uses **HTTPS**.

3. Security risks of using GET for authentication

- Password appears in the address bar and can be seen by shoulder-surfing or screenshots.
- It is saved in browser history, bookmarks and autocomplete suggestions.
- It is written to web server, proxy and CDN access logs (Figure 6).
- It leaks through the **Referer** header to external sites linked from the page.

- Links can be shared, cached or pre-fetched by crawlers and browsers, repeating the login request.

4. Best practices for secure login implementation

- Use **POST** over **HTTPS** only and enable HSTS.
- Store passwords hashed with a salt using bcrypt, scrypt or Argon2 – never in plain text.
- Add CSRF tokens, rate limiting or account lockout, and multi-factor authentication where possible.
- Issue session cookies with the **HttpOnly**, **Secure** and **SameSite** flags, and regenerate the session ID after login.
- Use parameterised queries, validate input and show generic error messages (“invalid username or password”).
- Never log passwords and use <input type="password"> for the password field.

```
<form action="/login" method="POST">
  <input type="text" name="user" required>
  <input type="password" name="pass" required>
  <button type="submit">Login</button>
</form>
```

Conclusion: GET is unsafe for credentials; POST over HTTPS together with hashing, CSRF protection and secure cookies is the correct design.

Question 4: HTML Code Inspection

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Missing closing tags

- **</h1>** is missing after “Welcome”.
- ** ** is missing after “This is a paragraph”.
- <img> is a void element, so it needs no closing tag in HTML5. In XHTML/XML it must be self-closed as .

2. Issues with the <img> tag

- No **alt** attribute – bad for accessibility (screen readers), SEO and shows nothing useful if the image fails to load; it is also a validation error.
- No **width** and **height** – the layout shifts when the image loads.
- The file path “image.jpg” is relative and must exist next to the page, otherwise a broken-image icon appears (as in Figure 8).
- The image is placed inside the paragraph, which is itself inside the heading (see the DOM below).

3. How browsers handle these errors

No error message is shown. The HTML5 parser recovers on its own: a start tag does **not** close an open <h1>, so the paragraph and the image become children of the heading, and everything is closed only at </body>. The result is that the paragraph text is displayed with heading styling (large and bold) instead of normal text.

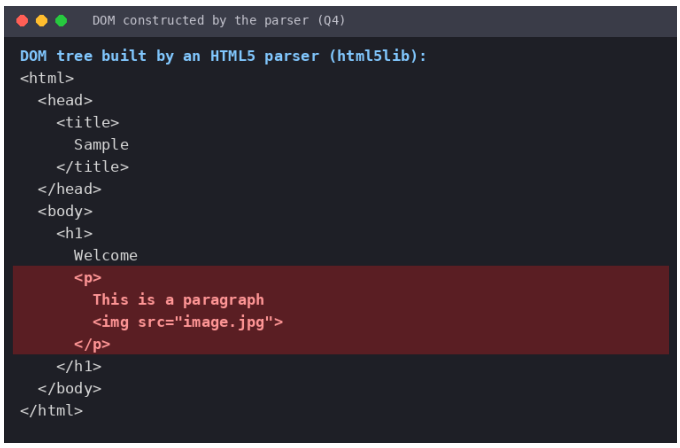


Figure 7: DOM built by the parser – <p> and <img> are nested inside <h1>

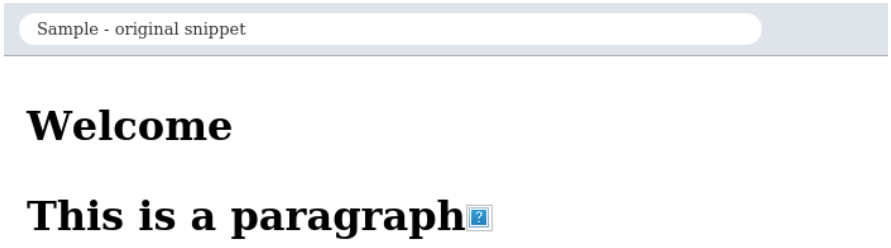


Figure 8: browser output of the original snippet – paragraph shown as a heading, broken image icon

4. Corrected version

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph</p>
<img src="image.jpg" alt="Sample image" width="120" height="80">
</body>
</html>
```

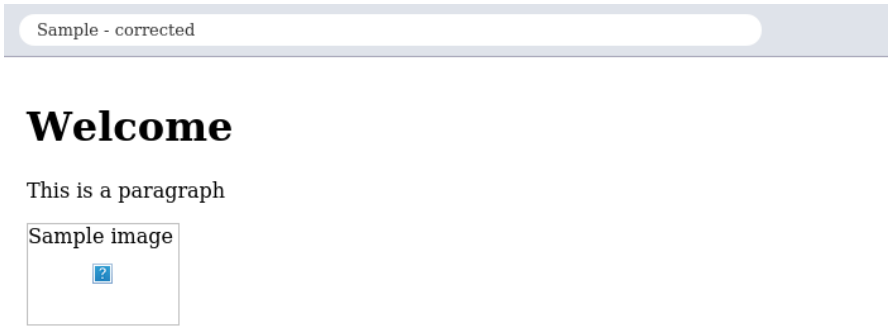


Figure 9: corrected output – normal paragraph; if image.jpg is missing the alt text is shown

Conclusion: the browser hides the mistakes but renders the page incorrectly. Closing every non-void tag and adding alt, width and height gives valid, accessible and predictable output.

Question 5: Browser-Server Communication Flow

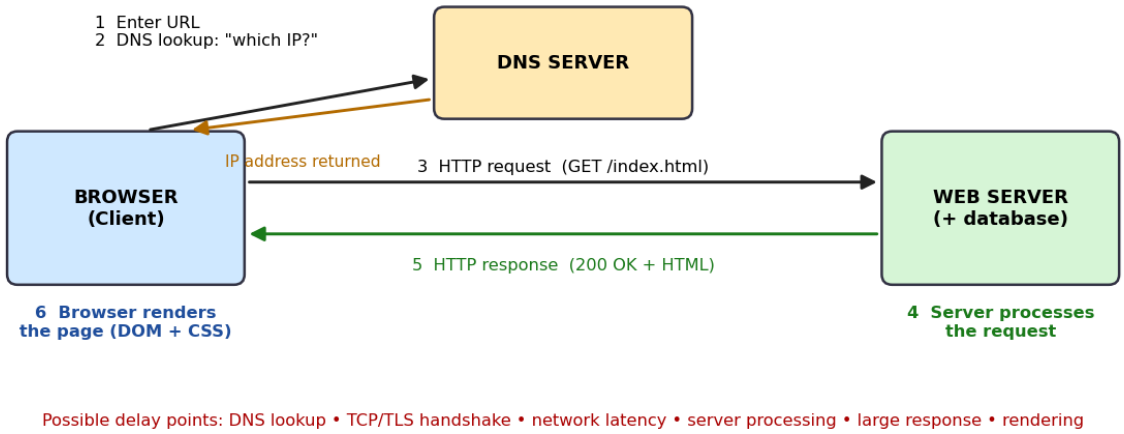


Figure 10: web request flow between browser, DNS server and web server

1. Interpretation of each step

Step	What happens
1. User enters a URL	The browser splits the URL into scheme (https), host (www.example.com) and path, and checks its cache and HSTS list.
2. DNS lookup	The domain name is converted to an IP address using the browser/OS cache or the DNS hierarchy (resolver → root → TLD → authoritative server).
3. HTTP request is sent	After the TCP (and TLS) handshake, the browser sends a request such as GET /index.html HTTP/1.1 with Host, cookies and other headers.
4. Server processes request	The web server routes the request, runs application code, queries the database and prepares the content.
5. Response is returned	The server sends the status line (200 OK), headers (Content-Type, Content-Length) and the body (HTML).
6. Browser renders page	The browser builds the DOM and CSSOM, fetches images/CSS/JS, then performs layout and paint to show the page.

2. Where delays can occur

- **DNS lookup** when the name is not cached, **TCP/TLS handshakes** and network latency.
- **Server processing** – slow code or slow database queries (the biggest delay in Figure 11).
- Large responses and many extra requests for images, CSS and JavaScript.
- Render-blocking CSS/JS in the browser.

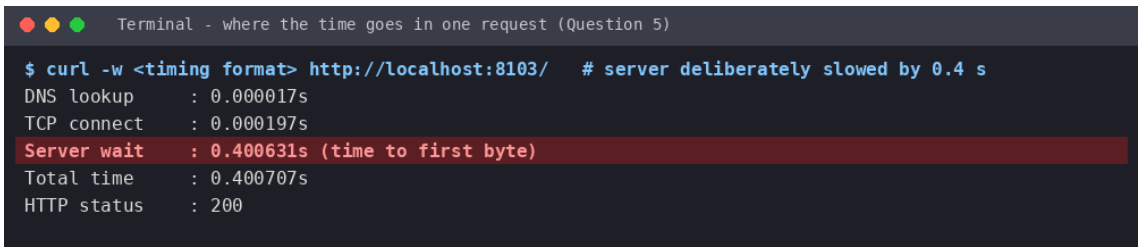


Figure 11: curl timing – DNS and connect are almost instant on localhost; server wait (0.4 s) dominates

3. Role of DNS in the process

Computers communicate using IP addresses, but people use names. DNS is the "phonebook of the Internet": it maps a domain name to the server's IP address, so the browser knows where to send the HTTP request. Results are cached at the browser, operating system and resolver for the record's TTL, which speeds up later visits. If DNS fails, the page cannot be reached at all, even though the web server is working. DNS can also direct users to the nearest server (CDN / load balancing).

4. Optimizations to improve performance

- DNS: use a fast DNS provider, reasonable TTLs, and <link rel="dns-prefetch"> / preconnect.
- Network: use a **CDN**, HTTP/2 or HTTP/3, keep-alive connections and TLS session reuse.
- Caching: send Cache-Control and ETag headers so repeat visits reuse content.
- Server: optimise database queries, add server-side caching and compress responses (gzip/Brotli).
- Browser: minify and bundle CSS/JS, load scripts with defer/async and lazy-load images.

Conclusion: a web page load is a chain of steps (DNS → connection → request → processing → response → rendering). The total time is the sum of all of them, so optimising the slowest step gives the biggest improvement.